Synthèse Bibliographique : Méthodes IA pour la synthèse des programmes à partir de la trace d'exécution

Bruno Fernandes Iorio Gildas De Michiel Imane Hibaoui

Decembre, 2025

Abstract

Ce document présente un état de l'art portant sur le domaine de la synthèse de programmes informatiques. La synthèse de programmes est un domaine de recherche qui vise à générer automatiquement du code à partir de spécifications données par l'utilisateur. L'objectif est de permettre à un système de produire un programme correct, ou du moins fonctionnel, sans intervention humaine. Les spécifications peuvent prendre diverses formes : des exemples d'entrées-sorties, des propriétés logiques du code attendu, des traces d'exécution ou encore des comportements observés.

Nous analysons et comparons plusieurs approches issues de la littérature récente. Nous concluons par une discussion critique sur les articles étudiés, ainsi que des perspectives de recherche liée à ce domaine.

Table des matières

1 Introduction **2**
- 1.1 Mots clés . . . 2
- 1.2 Bases de données consultées . . . 2

2 Approches dans la littérature **3**
- 2.1 Encodeur-Décodeur Neurale classique . . . 3
- 2.2 Transformer-based Neural Execution . . . 4
- 2.3 Grands modèles de langage (LLMs) . . . 5
- 2.4 Méthodes symboliques . . . 5

3 Discussion critique **6**

4 Conclusion **8**

1 Introduction

La synthèse de programmes est un processus automatique de production de code validant une spécification : historiquement, les approches neuronales de la synthèse de programmes traitent la problématique comme une génération de séquences (type modèles encodeur–décodeur). L'aptitude des approches s'est généralement révélée sur des bases de données simples (FlashFill [2] par exemple pour résoudre certains problèmes de transformation de chaînes). Toutefois, les modèles séquentiels traditionnels s'intéressent peu aux informations sémantiques qu'il est possible d'extraire de l'exécution du programme. Or, observer l'exécution (réelle ou simulée) du programme permet d'extraire des traces (sous forme de séquences d'états ou de sorties intermédiaires) qui prennent en compte le comportement dynamique du code. Ces traces d'exécution délivrent une information précieuse pour encadrer la synthèse de programmes,non plus seulement en tenant compte juste de la conformité attendue entre spécifications et code.

Dans les travaux récents que nous étudions ici, ces traces d'exécution permettent d'approfondir la recherche de synthèse de programmes. Nous avons retenus ici des contributions qui vont tant du recours à des usages neuronaux spécialisés ou non, que de l'approche classique de la synthèse symbolique guidée par la trace – qu'elles partagent pourtant dans le principe de travail consistant à intégrer l'information d'exécution au processus de l'ordinaire afin de retrouver un modèle le plus à même de produire les résultats souhaités. De l'apport de leurs usages, est ainsi à considérer cette diversité croissante des cas d'application allant de ce DSL (domain-specific languages) de robotique Karel qui se développe à des langages impératifs en général, en passant par les systèmes en temps réel et les logs de processus métier – le tout rapporté à ce qui est en rapport avec les modèles d'IA mobilisés : réseaux de neurones de spécialisation d'opérateurs en apprentissage ou non, LLM et autres algorithmes de synthèse syntaxique par exemple.

Dans ce qui suit, nous détaillons d'abord notre processus de recherche bibliographique (mots clés, bases de données explorées), puis nous présentons un tableau comparatif des travaux retenus, avant de fournir pour chacun une fiche d'analyse approfondie (contribution, méthodolo gie, résultats obtenus, limites et perspectives). Enfin, nous proposons une analyse critique transversale de ces approches et discutons des perspectives de recherche qu'elles ouvrent, sans chercher à apporter de solutions toutes faites.

1.1 Mots clés

Afin de constituer un corpus représentatif des travaux récents, nous avons mobilisé un ensemble étendu de mots-clés. Les recherches se sont fait en français comme en anglais pour couvrir la diversité des approches étudiées.

En français, nous avons notamment utilisé : *synthèse de programmes*, *génération automatique de code*, *traces d'exécution*, *raisonnement dynamique*, *exécution symbolique*, *analyse statique et dynamique*, *apprentissage profond*, *modèles neuronaux*, *modèles transformeurs*, *grands modèles de langage*, et *méthodes formelles de génération*.

Les termes anglais, plus largement employés dans la littérature, incluent : *program synthesis*, *execution traces*, *dynamic semantics*, *neural program synthesis*, *latent execution*, *execution-aware pretraining*, *trace-guided synthesis*, *symbolic execution*, *neuro-symbolic methods*, *large language models for code*, *code reasoning*, et *vulnerability detection*.

La combinaison de ces différents mots-clés a permis d'explorer efficacement les quatre grandes familles de travaux présentées dans cet état de l'art, en assurant une couverture équilibrée des dimensions neuronales, symboliques et hybrides.

1.2 Bases de données consultées

Pour chacune des requêtes, nous avons fait le choix de trouver des bases de données concernant les conférences et revues en informatique théorique, IA et en génie logiciel. Les sources suivantes

se sont révélées particulièrement utiles :

- ACM Digital Library & IEEE Xplore – Consultant à la recherche de références d'articles provenant des grandes conférences (ICLR, ICML, NeurIPS, ICSE, AAAI, etc.) et des revues IEEE/ACM. L'accès au texte entier étant parfois restreint, le titre, le résumé et les métadonnées fournies par ces bases ont permis d'identifier des travaux récents et d'extraire la référence complète.

- SpringerLink & Elsevier ScienceDirect – Ces deux bases ont été utilisées pour retrouver des articles dans des revues scientifiques (ex. Journal of Machine Learning Research). Elles ont aussi permis d'identifier des travaux supplémentaires.

- arXiv.org & Proceedings of Machine Learning Research (PMLR) – Ces deux plateformes constituent la source de prépublications accessibles librement, regroupant de nombreuses conférences en IA (ICML, NeurIPS, ICLR...) avec leurs versions auteurs. Elles ont été privilégiées pour récupérer les articles intégraux sélectionnés, afin de les lire intégralement et d'en extraire les citations adéquates. Ainsi, pour les articles NeurIPS 2020 et 2021 sélectionnés, nous avons opté pour arXiv.

- Google Scholar – Outil transversal de "qui cite qui" lorsque le format PDF n'était pas directement disponibles sur les sites officiels ; et d'autre part, pour la fonction "Cited by" permettant d'évaluer le rayonnement (nombre de citations, citations par des travaux récents) d'un article et de retrouver des travaux proches en suivant la liste de citations ou les articles semblables.

2 Approches dans la littérature

2.1 Encodeur-Décodeur Neurale classique

Les architectures encodeur-décodeur, fondées principalement sur des LSTM et des CNN sont apparues dans la synthèse de programmes entre 2018 et 2020. Dans ces modèles, l'encodeur traite les spécifications et le décodeur les transforme en jetons au fur et à mesure. Les travaux sur ce sujet montrent que les réseaux de neurones parviennent à apprendre à générer des programmes (e.g. robustFill [4]), mais ils rencontrent des problèmes par rapport à la correction sémantique, à l'interprétation des structures complexes et à la généralisation au-delà des exemples d'entraînement. Dans ce contexte, deux stratégies ont émergé : inférer une trace d'exécution intermédiaire ou générer des programmes candidats suivis d'une phase de correction.

La première est divisée en deux phases [12]. La première phase considère un réseau qui infère la trace complète à partir d'exemples d'entrées et de sorties comme par exemple le chemin pris par un robot dans une grille pour arriver a sa destination. Puis un second réseau qui produit ensuite le bon programme. Sur Karel benchmark, cela permet d'augmenter la performance de 77 à 81%. Les résultats baissent cependant lorsque les traces doivent être prédites plutôt que fournies. Une autre méthode génère des programmes pour les langages compilés comme C [3]. Elle apprend une exécution latente de capacité permettant de prédire des états intermédiaires pour des programmes incomplets. Sur un sous-ensemble restreint de C, elle atteint 55% contre 43% sans guidage, mais elle reste limitée à de simples opérations arithmétiques.

La seconde approche analyse les programmes candidats et utilise leurs traces pour corriger les erreurs. Le cadre "Synthesize-Execute-Debug" (SED) suit la même logique que le débogage humain [8]. Le synthétiseur produit des candidats de programmes qui sont exécutés en générant leurs traces. Ces traces sont utilisées par le débogueur qui les corrige. Ce cycle se répète jusqu'à obtenir une satisfaction des spécifications. Dans Karel, SED permet de réduire les erreurs d'au moins 7,5%. Une approche complémentaire consiste à exécuter le code dans un REPL et à évaluer les programmes partiels par une fonction de valeur apprise [6].

Ces approches partagent une idée commune : raisonner sur l'exécution plutôt que sur la syntaxe améliore la synthèse. L'inférence de traces, l'exécution latente et le débogage itératif entraînent des gains de l'ordre de 10 à 20 %. Cependant, plusieurs limitations existent. Les évaluations se concentrent principalement sur Karel, un DSL simple et déterministe. L'extension à des langages de grammaires plus complexes reste à faire. De plus, les LSTM peuvent avoir des difficultés avec les longues séquences et le contrôle de flux. Ce dernier point montre un signe que les modèles avec des structure encoder-decoder atteignent leur plafond d'efficacité, ce qui justifie l'exploration des modèles Transformers pré-entraînés.

2.2 Transformer-based Neural Execution

L'essor des architectures Transformer a complètement révolutionné le domaine d'analyse automatique du code en permettant la modélisation des dépendances à longue portée dans la structure des programmes. Dans cette section, nous discutons des travaux récents qui visent à incorporer les traces d'exécution dans l'apprentissage des Transformers, afin d'obtenir des modèles pouvant raisonner sur le comportement dynamique des programmes.

Le cadre TRACED [5] met en pratique cette démarche : il entraîne un Transformer à trois tâches d'apprentissage simultanées de génération de code source, de prédiction d'états de programme intermédiaires et d'estimation de la couverture de code. Cette contrainte de triple apprentissage oblige la représentation de la sémantique d'exécution par le modèle, aboutissant à une amélioration de 12.4% des prédictions de chemins d'exécution et de 25.2% des valeurs de variables par rapport aux modèles pré-entraînés statiquement. Ces avancées permettent de mieux détecter les vulnérabilités et de mieux identifier les clones de code.

Une seconde avancée majeure réside dans l'hybridation des représentations statiques et dynamiques, opérée par Concoction [13], un cadre appliquant des architectures Transformer pour fusionner les caractéristiques statiques courantes (l'arbre syntaxique abstrait, le graphe de flot de contrôle) et la trace d'exécution symbolique obtenue dynamiquement. Cette stratégie allie les motifs structurels et les comportements du programme pour établir une détection de vulnérabilités plus fine et plus exacte, car apprenant sur des projets C réels de grande taille, Concoction a révélé plusieurs vulnérabilités jusqu'alors inconnues. Simultanément, des travaux exploratoires [9] rendent compte des capacités émergentes des Transformers à internaliser la sémantique des programmes, puisque leur étude montre que des modèles n'ayant été exposés que sur des paires entrée-sortie finissent par développer spontanément des représentations internes des états intermédiaires d'exécution, atteignant une performance de 92,4% de précision en synthèse sur leur domaine. Cette maturité met en avant une certaine capacité des Transformers à modéliser le comportement dynamique du code source, ce qui constitue une voie prometteuse dans les configurations d'apprentissage non supervisé de la sémantique en programmation.

Des applications très pratiques telles que TraceFixer [1] témoignent déjà du succès de l'approche Transformer pour des tâches spécifiques comme la réparation de programmes. En fournissant à un modèle CodeT5 une trace d'exécution partielle jusqu'au point d'écart, le modèle produit des corrections qui semblent plus conformes et plus sensibles à la sémantique que les approches purement syntaxiques.

La fixation de la représentation des traces demeure une question ouverte : comment encoder des séquences d'états éventuellement nombreuses et structurellement complexes sous une forme digestible par les Transformers? Les problèmes d'échelle posent encore question. En effet, la plupart des validations sont faites sur des langages (DSL) ou des programmes de taille raisonnable, et le coût de la génération et du passage à l'échelle peut faire de ces méthodes d'apprentissage des méthodes de recherche de laboratoire à installer et maintenir hors recherche.

2.3 Grands modèles de langage (LLMs)

L'usage des grands modèles de langage constitue désormais un mode courant de synthèse de code. Toutefois, ces modèles s'avèrent souvent limités dans leurs capacités à raisonner finement sur une exécution, à expliquer l'émergence d'un bug ou à prévoir un comportement complexe. Les récents travaux cherchent à savoir si l'ajout de données d'exécution (traces, états intermédiaires, erreurs) permet d'augmenter les capacités de raisonnement, d'auto-réparation, ou d'interprétation du code.

Une première approche consiste à injecter les traces directement dans les prompts lors de l'inférence. Les auteurs de "Do Code Semantics Help?" [14] analysent l'influence de différentes formes d'information en provenance de l'exécution sur le fine-tuning supervisé et l'inférence. Les auteurs proposent un Trace Adapter qui se charge d'exécuter le code, d'en extraire les traces brutes, puis d'en réaliser une version normalisée, exploitable par un LLM. Les résultats obtenus montrent que les traces ne participent pas systématiquement à l'amélioration des performances. De plus, les modèles utilisant des traces d'exécution s'avèrent plus performants ou au moins aussi performants que ceux sans traces lors de la phase de fine-tuning. Lors de l'inférence, certaines stratégies basées sur le couplage de traces et de techniques de scaling obtiennent de légers gains lors de l'inférence, mais leurs résultats dépendent fortement du modèle utilisé et du type de trace ajouté.

Une seconde stratégie entraîne explicitement le LLM à raisonner avec les traces. NExT [10], par exemple, utilise une supervision où le modèle est entraîné à expliquer par étape les actions qu'exécute un programme dans le cadre du suivi de sa trace d'exécution. Le modèle génère un raisonnement rédigé d'une manière textuelle, décrivant l'évolution des variables ligne par ligne, puis localise la ligne où intervient le bug et enfin propose une correction. Des gains significatifs ont été observés sur les benchmarks de réparation de programmes : +26.1% sur Mbpp-R et +14.3% sur HumanEval-Fix. La qualité des raisonnements fournie par le modèle est bien meilleure que celle d'un LLM non entraîné, car le raisonnement prend bien soin de fournir des détails importants concernant la valeur des variables pendant l'exécution.

Ces résultats contrastés suggèrent en tout cas que les traces n'aideraient que si le modèle est explicitement entraîné à raisonner avec et non pas si l'on se contente de les ajouter aux données. Pourtant, plusieurs limites persistent. L'amélioration liée à l'utilisation de traces reste incertaine, et se compliquent avec le fait que NExT a uniquement été testé sur des programmes Python très courts possédant un seul bug. De plus, le coût lié à cet entraînement supplémentaire reste très élevé. Les traces deviennent difficilement exploitables même pour un LLM sur des programmes encore plus longs où l'objectif est d'accompagner l'analyse de programmes possédant plusieurs bogues. De manière globale, l'approche LLM présente par endroits un potentiel à long terme, mais des structurations plus précises sont nécessaires pour modéliser les informations dynamiques de manière appropriée.

2.4 Méthodes symboliques

Les méthodologies symboliques pour la synthèse de programmes se posent en complément de solutions sur la base d'approches strictement neuronales. En effet, suivant des attendus formels tels que la syntaxe guidée par l'interprétation (SyGuS) ou bien les espaces de versions, elles permettent de garantir la génération de programmes bien structurée. Des travaux récents proposent d'incorporer les traces d'exécution pour guider la recherche symbolique ou d'associer des méthodes formelles et des réseaux de neurones dans des résolutions hybrides.

Une première approche utilise les traces de récursion pour guider la synthèse inductive. C'est à l'image de SyRup [15] qui propose d'élargir l'espace de recherche avec des traces de récursion et des suites d'appels récursifs des programmes. Une nouvelle algèbre des espaces de versions (VSA) permet de bien représenter et de manipuler les paires ainsi introduites (trace, programme) sous conditions de compatibilité.

Cette méthode permet de résoudre l'ambiguïté intrinsèque de la synthèse à partir d'exemples : les traces font contrainte à la fois sur le résultat final mais aussi sur le comportement intermédiaire du programme. L'outil SyRup, qui met en œuvre cette approche, requiert moins d'exemples pour atteindre un taux de succès comparables aux synthétiseurs existants. De plus, cette méthode est également moins sensible à la qualité des exemples fournis.

Une seconde stratégie se trouve sur la synthèse à partir de traces partielles de systèmes à effets de bords. Syren [7] est à notre connaissance la première technique capable de synthétiser des programmes composant à la fois des fonctions à effets de bord, des fonctions pures, et des structures de contrôle à partir de traces partielles connues. Ces traces partielles ont pour particularité de ne pas contenir les appels aux fonctions à effets de bord. Ceci concerne en particulier la synthèse de scripts d'automatisation, à partir de logs d'appels API ou de traces d'appels système. Contrairement aux autres méthodes qui requièrent des contre-exemples, Syren généralise uniquement à partir d'exemples positifs, tout en utilisant des métriques de coût pour pallier au risque de sur-généralisation. La méthode consiste à combiner des réécritures optimisantes et un synthèse guidée par la syntaxe (SyGuS) pour inférer les fonctions pures cachées entre les appels observés. Enfin, LamdbaBeam [11] propose un algorithme de recherche capable de construire des fonctions lambda arbitraires utilisant uniquement comme composants des opérations d'un DSL. L'originalité de ce travail repose sur l'utilisation de représentations vectorielles sémantiques du comportement d'exécution des fonctions lambda. Les signatures sémantiques, qui sont obtenus par exécution, permettent de guider un réseau de politique neuronal capable de décider quelles lambdas construire, dans le cadre d'une recherche de programme. Pour ensuite les passer en argument à des fonctions d'ordre supérieur pour réaliser des calculs itératifs. Dans le domaine de la manipulation de listes d'entiers, LambdaBeam surpasse les méthodes purement neuronales, purement symboliques et celles utilisant les LLMs.

Toutes ces approches reposent sur un principe commun : les traces d'exécution (traces de récursion, logs d'appels système ou signatures sémantiques) permettent de contraindre et de guider efficacement l'espace de recherche symbolique. Les méthodes symboliques pures (SyRup, Syren) garantissent la correction du code, mais ne peuvent être employées que sur des DSLs ou des domaines restreints. L'approche hybride de LambdaBeam montre que lorsque guidée par l'exécution, une politique neuronale peut être mise en œuvre pour étendre les capacités de la synthèse symbolique, même pour les programmes faisant appel à des fonctions d'ordre supérieur. Cela dit, le problème reste entier pour la mise à l'échelle de toutes ces méthodes, car elles sont très dépendantes de la qualité et de la complétude des traces.

3 Discussion critique

Les quatre approches s'appuient sur des hypothèses différentes en ce qui concerne le rôle de l'exécution dans la compréhension du code. Ces différences ont pour conséquence des coûts computationnels et des niveaux de performance distincts.

L'Approche 1 (Encodeur-Décodeur) repose sur l'idée qu'une exécution réelle, même partielle, fournit la meilleure forme de guidance. Cette hypothèse rend l'inférence coûteuse, car elle nécessite à la fois plusieurs modèles spécialisés et un moteur d'exécution fiable. L'Approche 2 (Transformers) suppose qu'un modèle peut apprendre à approximer une exécution à partir de traces annotées. L'entraînement du modèle devient coûteux, car il exige un volume important de données dynamiques. L'Approche 3 (LLMs) suppose que les modèles de langage peuvent tirer parti des traces d'exécution sans les simuler explicitement, soit via une supervision coûteuse, soit via une intégration directe de traces brutes, dont l'efficacité reste limitée. Enfin, l'Approche 4 (Symbolique) repose sur des méthodes formelles garantissant la correction par construction, utilisant les traces comme contraintes pour guider la recherche symbolique. Cependant, cette recherche possède une scalabilité limitée par l'explosion combinatoire de l'espace de recherche.

En termes de robustesse et de généralisation, les contrastes sont tout aussi marqués. L'Approche

1 (Encoder-Decodeur), bien que très fiable dans son domaine cible, ne se généralise pas ou peu au-delà de petits DSL tel que Karel. En effet, son mécanisme dépend étroitement de la structure du langage et de la disponibilité d'un interpréteur efficace. L'Approche 2 (Transformers) offre une meilleure généralisation, notamment sur des langages plus complexes comme C. Cependant, cette méthode est uniquement efficace sur des programmes relativement courts. L'Approche 3 (LLMs) présente en théorie le potentiel de généralisation le plus élevé. En effet, les grands modèles peuvent acquérir des compétences à partir de larges ensembles de traces ou de raisonnements supervisés. Cependant, cette généralisation est fortement limitée par les stratégies d'entraînement employées, et les performances varient fortement selon la façon dont l'information est fournie au modèle. L'Approche 4 (Symbolique) offre des garanties formelles de correction, mais reste confinée à des domaines spécifiques : fonctions récursives pour SyRup, scripts d'API pour Syren, manipulation de listes pour LambdaBeam.

Les quatre approches se distinguent également par la manière dont elles utilisent l'exécution elle-même. L'Approche 1 (Encoder-Decoder) s'appuie sur des traces directement obtenues à partir du code. L'Approche 2 (Transformers) se base au contraire sur une exécution prédite par le modèle. Cette méthode est plus flexible, mais aussi plus vulnérable aux erreurs d'approximation. L'Approche 3 (LLMs) ne simule pas et n'exécute pas réellement le programme. Elle raisonne plutôt sur les traces du programme. Le modèle n'a pas besoin d'interpréteur, mais il nécessite une formulation de traces adaptée aux capacités des LLM. L'Approche 4 (Symbolique) utilise les traces comme contraintes formelles. Ces traces de récursion orientent la synthèse inductive et les logs d'appels système pour inférer les fonctions cachées, ou les signatures sémantiques d'exécution.

Ces travaux montrent que les quatre approches se complètent. L'Approche 1 (Encoder-Decoder) est la plus efficace dans des environnements restreints. Les Approches 2 (Transformers) et 3 (LLMs) semblent plus versatiles sur le plan technique, et plus prometteuses à long terme. L'Approche 4 (Symbolique/Hybride) garantit la correction par construction et montre, via LambdaBeam, que la combinaison neuronale-symbolique peut surpasser les méthodes purement neuronales ou purement symboliques. Cependant, chacune présente des limites structurelles. L'Approche 1 (Encoder-Decoder) a du mal à se généraliser en dehors de son cadre. L'Approche 2 (Transformers) dépend fortement de données annotées difficiles à produire. L'Approche 3 (LLMs) reste instable lorsque les traces sont intégrées de manière brute. L'Approche 4 (Symbolique/Hybride) a du mal à se généraliser vers des langages plus complexes et dépend de la complétude des traces disponibles.

Ces résultats indiquent que l'enjeu central n'est pas seulement d'utiliser l'exécution, mais aussi de déterminer quelle forme d'information dynamique peut réellement améliorer le raisonnement des modèles de génération de code.

Table 1: Comparaison des Méthodes

Critère	**Encodeur-Décodeur**	**Transformers**	**LLMs**	**Symbolique**
Articles	[3, 6, 8, 12]	[1, 5, 9, 13]	[10, 14]	[7, 11, 15]
Coût d'entraînement	Modéré	Élevé	Très élevé / Nul	Nul
Coût d'inférence	Élevé	Faible	Modéré	Variable
Généralisation	Faible (DSLs)	Moyenne	Potentiellement élevée	Faible
Garantie de correction	Non	Non	Non	Oui
Amélioration typique	10-20%	12-25%	0-26%	N/A
Domaines validés	Karel, C restreint	C, Python, vulnérabilités	Python (courts)	Fonctionnel, APIs, listes

4 Conclusion

Cette synthèse bibliographique s'est intéressée à la manière dont les informations issues de l'exécution peuvent améliorer la synthèse et la compréhension de programmes. Quatre grandes approches ont été analysées. Ces approches offrent des visions différentes de l'usage des informations dynamiques. Leurs performances mettent en évidence leurs forces, mais aussi les limites liées à leur utilisation.

Les travaux étudiés montrent d'abord que l'intégration de traces améliore la capacité des modèles à générer ou corriger du code, mais uniquement sous certaines conditions. La première approche fournit les gains les plus stables dans des environnements DSL tels que Karel. Cependant, ses mécanismes dépendent d'un interpréteur spécifique et d'un contexte restreint. La deuxième approche se révèle plus flexible. En effet, les Transformers peuvent apprendre un comportement d'exécution sans le simuler. Cette particularité permet une meilleure généralisation sur des langages de programmation ou des tâches de sécurité. Cependant, elle souffre d'un coût d'entraînement élevé et d'une dépendance à des ensembles de traces annotées difficiles à produire à grande échelle. La troisième approche semble plus prometteuse sur le long terme. Les LLM entraînés à raisonner sur des traces, développent une meilleure compréhension du comportement des programmes. Cette méthode montre que l'usage de traces devient réellement efficace lorsque le modèle apprend à raisonner avec elles. Néanmoins, cette supervision est coûteuse. De plus, l'approche n'est pas adaptée à des programmes longs ou complexes.

La quatrième approche offre des garanties formelles de correction, mais reste confinée à des domaines spécifiques et a du mal à se généraliser à des langages plus complexes. Tout les articles étudiés montrent que les éléments liés à l'exécution du programme sont des atouts indispensables. Il sert à améliorer les résultats obtenus pour la synthèse et la compréhension de programme informatique. Cependant, son utilisation exige de créer des méthodes adaptées aux modèles utilisés. Pour finir, cet état de l'art soulève un point important dans la recherche sur le domaine : comment structurer les traces d'exécution pour que les modèles de langage puissent analyser et raisonner sur le comportement d'un programme ?

References

[1] Islem Bouzenia, Yangruibo Ding, Kexin Pei, Baishakhi Ray, and Michael Pradel. Tracefixer: Execution trace-driven program repair, 2023.

[2] José Cambronero, Sumit Gulwani, Vu Le, Daniel Perelman, Arjun Radhakrishna, Clint Simon, and Ashish Tiwari. Flashfill++: Scaling programming by example by cutting to the chase. *Proc. ACM Program. Lang.*, 7(POPL), January 2023.

[3] Xinyun Chen, Dawn Song, and Yuandong Tian. Latent execution for neural program synthesis. In *Proceedings of the 35th International Conference on Neural Information Processing Systems*, NIPS '21, Red Hook, NY, USA, 2021. Curran Associates Inc.

[4] Jacob Devlin, Jonathan Uesato, Surya Bhupatiraju, Rishabh Singh, Abdel rahman Mohamed, and Pushmeet Kohli. RobustFill: Neural program learning under noisy I/O. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 990–998. PMLR, 06–11 Aug 2017.

[5] Yangruibo Ding, Ben Steenhoek, Kexin Pei, Gail Kaiser, Wei Le, and Baishakhi Ray. Traced: Execution-aware pre-training for source code, 2023.

[6] Kevin Ellis, Maxwell Nye, Yewen Pu, Felix Sosa, Josh Tenenbaum, and Armando Solar-Lezama. Write, execute, assess: Program synthesis with a repl, 2019.

[7] Margarida Ferreira, Victor Nicolet, Joey Dodds, and Daniel Kroening. Program synthesis from partial traces. *Proceedings of the ACM on Programming Languages*, 9(PLDI):1642–1665, June 2025.

[8] Kavi Gupta, Peter Ebert Christensen, Xinyun Chen, and Dawn Song. Synthesize, execute and debug: Learning to repair for neural program synthesis, 2020.

[9] Charles Jin and Martin Rinard. Emergent representations of program semantics in language models trained on programs, 2024.

[10] Ansong Ni, Miltiadis Allamanis, Arman Cohan, Yinlin Deng, Kensen Shi, Charles Sutton, and Pengcheng Yin. Next: Teaching large language models to reason about code execution, 2024.

[11] Kensen Shi, Hanjun Dai, Wen-Ding Li, Kevin Ellis, and Charles Sutton. Lambdabeam: Neural program search with higher-order functions and lambdas, 2023.

[12] Richard Shin, Illia Polosukhin, and Dawn Song. Improving neural program synthesis with inferred execution traces. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, NIPS'18, page 8931–8940, Red Hook, NY, USA, 2018. Curran Associates Inc.

[13] Huanting Wang, Zhanyong Tang, Shin Hwei Tan, Jie Wang, Yuzhe Liu, Hejun Fang, Chunwei Xia, and Zheng Wang. Combining structured static code information and dynamic symbolic traces for software vulnerability prediction. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE '24, New York, NY, USA, 2024. Association for Computing Machinery.

[14] Jian Wang, Xiaofei Xie, Qiang Hu, Shangqing Liu, and Yi Li. Do code semantics help? a comprehensive study on execution trace-based information for code large language models, 2025.

[15] Yongwei Yuan, Arjun Radhakrishna, and Roopsha Samanta. Trace-guided inductive synthesis of recursive functional programs. *Proc. ACM Program. Lang.*, 7(PLDI), June 2023.